第 4 章 大模型的心脏：Transformer 架构

在大语言模型（Large Language Models, LLMs）重塑医疗 AI 版图的今天，Transformer 架构无疑是其最为核心的动力来源。医疗文本具有高度的专业性、信息高密度以及极强的上下文依赖特征——一份长达数十页的住院病程记录中，患者早期的用药史可能直接决定了当前的诊断逻辑。

传统模型如循环神经网络（RNN）或长短期记忆网络（LSTM）在处理这类文本时，往往受限于“序列依赖”：模型必须按顺序读取信息，且随着序列增长，早期的临床细节会在复杂的梯度计算中逐渐“稀释”。Transformer 引入的自注意力机制（Self-Attention），彻底打破了这一瓶颈。它允许模型在处理当前词汇时，能够瞬间捕捉到文档中任何位置的相关信息。这种并行处理能力，不仅确保了复杂临床证据的可追溯性，更在大规模医学知识表征中展现了极高的稳健性。

4.1 注意力机制（Self-Attention）

自注意力机制可以被直观地理解为在海量医疗数据中“划重点”。在阅读一份包含主诉、现病史、实验室检查及影像学表现的复杂病历时，临床医生并不会等权对待每一个字，而是会根据“主诉”中的关键症状去病历的其他部分寻找支持证据。自注意力机制在数学上实现了这种“选择性关注”。

其核心逻辑基于三个向量：查询（Query, Q）、键（Key, K）和值（Value, V）。这三个向量是通过对原始输入进行线性变换得到的，赋予了模型从不同维度审视医疗文本的能力：

- **Query (Q)**：代表当前正在处理的术语。例如，当模型扫描到“头痛”这个词时，Q 向量就携带了“寻找头痛原因或关联症状”的指令。
- **Key (K)**：代表文本中所有术语的特征索引。例如，“发热”、“血压”、“脑 CT 结果”各自拥有不同的 K 向量。
- **Value (V)**：代表各术语所携带的具体医学信息内容。

计算过程遵循以下缩放点积注意力公式：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

其中，QK^T 的运算实际上是在计算“查询”与“键”之间的匹配度。如果 Q（头痛）与某个 K（恶心）在特征空间高度相关，点积值就高。随后，通过 $\sqrt{d_k}$ 进行缩放以维持梯度稳定，并经过 Softmax 归一化，将分值转化为 0 到 1 之间的概率权重。最后，模型根据这些权重对 Value 进行加权求和，输出一个聚合了上下文语义的特征向量。

[输入 Token 序列] → [患者] [主诉] [胸痛] [伴] [呼吸] [困难]

Step 1: 向量映射 → 每个 Token 映射为 Q, K, V 高维空间

Step 2: 评分计算 → “胸痛”与“呼吸困难”的相关性远高于“伴”

Step 3: 权重分配 → 通过 Softmax 赋予“呼吸困难”极高的权重系数

Step 4: 语义聚合 → “胸痛”的最终向量包含“急性心肺症状”的复合语义

图 4.1: 自注意力机制在医疗文本中的特征提取逻辑

在医疗场景落地中，这种机制的优势在于处理“跨度依赖”。例如，当病历开头提到“患者有青霉素过敏史”，而在文末出现“处方：阿莫西林”时，自注意力机制能跨越数千字的干扰，在两者之间建立强连接。这种能力使得 AI 在进行药物配伍禁忌检查或构建临床决策支持（CDSS）系统时，能精准聚焦于关键证据链。（注：以上示例仅用于技术原理解释，非临床诊断建议；实际应用中必须遵循相关医疗器械监管合规要求。）

4.1.1 KV Cache 与多头注意力的权衡

在真实的医疗 AI 工程落地中，纯粹的注意力计算会面临巨大的计算开销。现代系统普遍采用“多头注意力（Multi-Head Attention）”，即同时运行多个 $Q/K/V$ 计算支路。在处理一份影像报告时，一个“头”可能关注解剖位置，另一个“头”关注病灶性质（如结节、浸润）。

为了提升推理效率，现代 LLM 系统常引入 KV Cache 技术。在生成式对话中，模型每产生一个新词，并不需要重新计算前文所有词的 K 和 V 向量，而是直接从缓存中读取。此外，为了进一步压缩显存占用，诸如分组查询注意力（GQA）或 FlashAttention 等算法被广泛应用。这些手段解决了医疗实时问答系统在高并发下的延迟问题。本质上，这些工程优化仍是注意力机制在硬件底层的高效实现，其目标是在不损失医学推理精度的前提下，最大限度降低显存带宽的压力。

- **风险点：证据不可追溯性**——注意力权重虽然提供了某种“解释性”，但在多层深度网络中，单一权重的指向并不等同于医学上的金标准因果关系。
- **风险点：长文本退化**——尽管架构支持长上下文，但随着输入增长，注意力矩阵的稀疏性可能导致模型忽略一些低频但关键的实验室指标（如罕见的基因突变分型）。

4.2 解码器与编码器：主流模型流派（GPT vs. BERT）

Transformer（Transformer）架构通常由编码器（Encoder）与解码器（Decoder）两类模块构成：编码器更像“读懂并压缩”，负责把输入序列整合成语义表征；解码器更像“按约束写出”，在历史上下文条件下逐步生成输出。二者的关键差异在于注意力的可见性约束：编码器常采用双向（bidirectional）建模，能够同时利用左右上下文完成更充分的语义理解；解码器则采用因果掩码（causal mask），只能基于已出现的内容自回归生成，天然适配问答、摘要与报告撰写等生成任务。围绕这两类模块的组合方式（Encoder-only、Decoder-only、Encoder–Decoder），医疗 AI 工程实践逐渐形成两条典型技术路线：以 BERT（BERT）为代表的“理解型”流派，擅长检索、分类、实体抽取与结构化信息提取等需要稳定表征与可控输出的场景；以及以 GPT（GPT）为代表的“生成型”流派，更擅长多轮交互、自然语言总结与报告生成等需要流畅表达与上下文续写的场景。

4.2.1 编码器流派：医学语义的深度理解

编码器流派（Encoder-only）通常采用双向注意力（Bidirectional Attention）：在计算序列中任意位置的表示时，模型可以同时“看到”该词元（token）左侧与右侧的上下文信息。直觉上，这更像临床中的“病案复习”：医生在合上病历前，会把主诉—现病史—查体—检验影像—处置记录来回对照，确认每个结论都能在前后文中找到支撑。双向建模带来的优势是：它更擅长理解否定、限定、时间线与指代这类医学文本高频且高风险的语义结构。例如“否认胸痛”“既往青霉素过敏”“术后第 3 天体温较前下降”这些信息，往往需要同时利用前后文才能还原真实含义；若只看单侧上下文，模型容易把“否认”漏掉或把时间线搞反，从而影响后续抽取与分类的可靠性。

从机制上看，编码器的双向注意力意味着注意力矩阵不受因果掩码（Causal Mask）限制，任意位置 i 在更新表征时都可以对所有位置 j 分配权重，这使得模型可以在同一层内建立全局语义对齐。编码器流派最经典的训练目标是掩码语言模型（Masked Language Model, MLM），即随机遮盖部分词元，要求模型根据其余上下文预测被遮盖的词元：

$$\mathcal{L}_{\mathrm{MLM}}(\theta) = -\sum_{i \in \mathcal{M}} \log p_\theta\big(x_i \mid x_{\setminus i}\big)\,.$$

这一目标在医学语料上有很强的“语义约束”效果：模型必须学会从上下文推断出被遮盖的医学术语、检查项目、剂量单位与解剖部位等信息，从而形成更细粒度、更稳定的语义表示。举例来说，句子“患者_ 青霉素过敏史”中被遮盖的是“否认/有”，模型需要结合上下文判断；又如“给予_ 0.4 mg 舌下含服”，模型要依赖用药习惯与剂量单位来恢复药名。这类训练并不会让模型“学会诊断”，但会让它对医学文本的结构、搭配与约束更敏感，进而适合做严谨的理解型任务（非临床建议/需遵循合规）。

在工程落地中，编码器模型常被放在医疗系统的“后台结构化引擎”位置：它不直接给出临床处置建议，而是把非结构化文本转换为可检索、可统计、可审计的结构化信号。典型任务包括医疗命名实体识别（Named Entity Recognition, NER）、关系抽取（Relation Extraction）、片段级分类（span classification）与检索向量构建（dense retrieval embedding）。例如，从病历中抽取手术名称、解剖部位、用药频次、过敏史、家族史等字段时，编码器可以为每个词元提供上下文化表示，再通过序列标注头（如 BIO 标签）输出实体边界；在 ICD-10（International Classification of Diseases, 10th Revision）编码中，编码器可将诊断描述映射到类别空间，作为半自动编码的候选排序模块；在质控场景中，它也可用于识别“诊断与证据不一致”“关键字段缺失”“否定信息未覆盖”等文本模式，辅助人工审阅而非替代审阅。

以 ClinicalBERT（ClinicalBERT）这类医学领域预训练编码器为例，其常见用法并不是“端到端生成结论”，而是作为可控的组件嵌入流水线：先对文本进行脱敏与分段，再做实体抽取与编码分类，最后将结构化结果写入审计日志并供上层系统检索调用。这样做的好处在于输出形式更可控、评测更标准化（如实体级 F1、编码准确率、召回率），也更容易设置合规边界与人工复核点。需要强调：无论抽取或分类结果多么“像临床结论”，都应被视为信息处理与辅助标注，不得直接作为诊断建议或处置依据；上线系统必须遵循数据合规、权限控制与责任分工（非临床建议/需遵循合规）。

4.2.2 解码器流派：临床逻辑的连续生成

解码器流派（Decoder-only）的核心特征，是在自注意力（Self-Attention）中引入因果掩码（Causal Masked Attention）：当前位置 i 只能关注到历史位置 $1 \ldots i$，而不能访问未来位置 $i+1 \ldots n$。直觉上，这相当于给模型加上一条“时间箭头”：它必须沿着输入的时间线逐步推进，不能像编码器那样把整段文本读完再回头修正。这种约束与临床书写和问答的工作方式相似——医生在撰写病程记录、出院小结或解释检查结果时，往往是在已知事实与既有上下文基础上组织下一句话：先交代主诉与现病史，再补充检查与处置，最后形成结构化的总结陈述。需要强调的是，这里所谓“模拟临床逻辑”，指的是文本生成的组织方式与信息递进的形式，并不意味着模型具备临床判断资格或可替代诊疗决策（非临床建议/需遵循合规）。

从机制上看，因果掩码可以理解为在注意力得分矩阵中把未来位置设为不可见（近似为 $-\infty$），使得 softmax 后这些位置权重为 0。这样一来，解码器在生成第 i 个词元时，只能利用前文来计算上下文表示。其训练目标通常是自回归预测（Autoregressive Prediction），即最大化整段序列出现的概率：

$$P(x) = \prod_{i=1}^{n} P(x_i \mid x_1, \ldots, x_{i-1}).$$

训练时常用“教师强制”（teacher forcing）：模型在预测 x_i 时输入真实的历史 $x_{<i}$，并用交叉熵（cross-entropy）优化下一词元的对数似然。这个细节很重要：训练阶段模型总是站在“正确历史”之上，而推理阶段模型必须依赖自己刚生成的历史继续往下写，这会带来“暴露偏差”（exposure bias）——一旦早期生成出现偏离，后续就可能沿着错误轨迹越写越顺。医疗场景对这一点格外敏感，因为语言越流畅，越容易造成“可信错觉”，把措辞专业误当成事实可靠。

在医疗应用中，GPT（GPT）类解码器架构之所以成为前端交互的常见选择，原因在于它天然擅长把多源信息组织成连贯输出：例如将检验单中的多项指标用自然语言解释给患者或基层医护，将病程要点整理为出院小结的结构化段落，或在多轮对话中维持上下文一致性（如代词指代、时间线延续、问题聚焦）。但这些优势建立在“连续生成”的范式上，也意味着工程侧必须主动设计约束：一方面让模型有据可依（例如将病历片段、检验摘要、用药清单以明确的证据区块提供给模型）；另一方面让模型按规输出（例如限定输出为 JSON（JavaScript Object Notation）字段、固定小结模板或可审计的段落结构），避免把生成当作自由写作。举例来说，在解释检验单时，系统可以要求模型逐条引用“指标名–数值–参考范围–原文位置”，再生成面向非专业读者的解释；这类“先对齐证据、再生成语言”的流程，通常比直接让模型自由回答更稳健，也更利于事后审计（非临床建议/需遵循合规）。

因此，解码器流派在医疗落地的关键不在“能写多像”，而在“能否被约束、能否可追溯”。工程实施中常

见的抓手包括：记录输入证据片段与生成参数（温度 temperature、top-p 等）、对输出做结构化校验与敏感内容过滤、设置拒答与转人工策略、以及在高风险场景引入检索增强生成（Retrieval-Augmented Generation, RAG）与重排序（reranking）来降低凭空编造的概率。归根结底，解码器的强项是把已有信息组织成语言；要让它在医疗系统里“可用且可控”，必须把证据链、格式约束与审计机制一并作为系统能力来建设，而不是只关注模型本身。

Encoder-only（理解型）：全双向交互。输入 [咳嗽] [咳痰] [发热]，模型同时理解三者关联。→ **输出：肺炎相关标签（分类，示例）**
Decoder-only（生成型）：单向约束。输入 [患者]，逐步生成 [可能] [需要] [进一步评估] 等连贯文本。→ **输出：说明性文本（生成，非临床）**
Encoder–Decoder（混合型）：编码端理解病历要点，解码端在约束下生成目标格式内容。→ **输出：英文摘要/结构化小结（生成，示例）**

图 4.2: 不同 Transformer 流派在医疗任务中的功能划分（示例仅用于技术说明，非临床建议/需遵循合规）

4.2.3 工程实践：位置编码与长文本的处理艺术

由于 Transformer（Transformer）的注意力计算本身对“顺序”不敏感——如果只看词元（token）之间的相似度，模型可能把“先出现发热后出现咳嗽”和“先咳嗽后发热”当成差不多的语义——因此必须引入位置编码（Positional Encoding），把“第几个词元、相隔多远、谁在谁之前”这类信息注入到表示中。通俗地说，位置编码相当于给病历里的每一句话加上时间轴刻度，让模型在做证据对齐时不会把顺序线索弄丢。

在医疗长文本场景里，绝对位置编码（absolute positional encoding）常见的问题是“长度天花板”：模型训练时见过的最大长度有限，超过窗口后，后半段的位置信号要么缺失、要么分布外，容易导致注意力漂移与时间线混乱。现代工程实践更常用旋转位置嵌入（Rotary Position Embedding, RoPE）。RoPE 的核心思想是把位置差异编码为向量空间中的“旋转角度”，使得注意力在计算 $q_i^\top k_j$ 时自然带上相对位移信息，因此模型更容易学习“相对距离与先后关系”，而不是死记绝对索引。工程含义是：即便训练主要覆盖 2k 左右的长度，推理时通过合理的 RoPE 扩展与窗口策略，模型在更长病历中仍有机会保持时间线一致性，例如把“既往史”“入院后用药”“复查结果”按顺序关联起来。需要注意的是，这并不意味着长度外推永远可靠：超长输入下仍可能出现远距离依赖衰减、关键信息被稀释等现象，因此系统通常还会配套滑动窗口（sliding window）、分段摘要与检索增强（Retrieval-Augmented Generation, RAG）等手段，保证关键证据在窗口内可被稳定关注（非临床建议/需遵循合规）。

- **风险点：生成幻觉（Hallucination）**——解码器流派倾向于生成统计学上最可能的下一个词，而非医学事实。这可能导致模型编造出不存在的药物剂量或检查结果。
- **风险点：累计误差**——在长程生成中，前文一个微小的医学术语偏差（如将“阳性”误写为“阴性”）会被单向约束机制无限放大，导致后续结论完全背离事实。